



REAL-TIME SYSTEMS

UNIT 1 — MINI PROJECT REPORT

SIMULATION, MEASUREMENT-BASED WCET ESTIMATION, AND REAL-TIME JITTER ANALYSIS

COMPUTER ENGINEERING YEAR IV

MEMBERS:

- **Blandon ASHIMIRWE 223026675**
- **Janvier NSHIMIYIMANA 223026679**
- **Gilbert HIRWA NTWALI 223026689**

**PREPARED FROM THE REAL-TIME SYSTEMS
— MINI PROJECTS, UNIT 1 LAB MANUAL
SEPTEMBER 2026**

Table of Contents

Table of Contents	2
1. Analysis of the Lab Manual	3
Observations about the manual and the choices made	3
Manual's expected observations versus what was measured	4
Files delivered	5
2. Mini Project 1.1: Real-Time Task Timing Behavior	6
2.1 Method.....	6
2.2 Step 2: execution time in isolation	6
2.3 Steps 3-9: periodic run.....	7
2.4 Lab tasks	9
2.5 Step 9 summary	12
3. Mini Project 1.2: Measurement-Based WCET Estimation Tool.....	13
3.1 Tool design.....	13
3.2 Comparison table (microseconds)	13
3.3 Histograms	14
3.4 Discussion	15
4. Mini Project 1.3: Real-Time Jitter Analyzer	17
4.1 Setup.....	17
4.2 Results table.....	17
4.3 Sweep plots.....	19
4.4 Jitter growth without a drop in hit fraction	20
4.5 Discussion: mapping the disturbances to Section 1.6	23
References	24
GitHub Link.....	25

1. Analysis of the Lab Manual

The three mini projects turn the timing vocabulary of Unit 1 into things you can measure. Each one adds a layer: 1.1 measures a single periodic task, 1.2 asks how trustworthy a measured worst-case execution time is, and 1.3 isolates jitter and shows that it is a separate property from meeting deadlines.

Project	Core concept	What must be built	Key deliverable
1.1 Task timing	Release time, deadline, response time, WCET, jitter, laxity; hard/firm/soft cost functions (Sections 1.2, 1.3, 1.7)	A variable-time job function, periodic release loop, timestamp instrumentation, hit fraction, three utility sequences, two plots, four lab tasks	Code, two plots, one-page report with Step 9 summary and answers to the four lab tasks
1.2 WCET tool	Measurement-based WCET estimation, its unsoundness, safety margin (Sections 1.7, 1.8)	A reusable harness for any function and input generator, statistics, margin, histogram, at least three targets with random and adversarial strategies	Code, histogram per target, comparison table, one-page discussion
1.3 Jitter analyzer	Release versus completion jitter; non-determinism sources (Sections 1.3, 1.6)	Drift-free loop plus three independent disturbances (background load, random stall, execution-time variability), sweeps, results table	Code, sweep plots, combined plot, results table, one-page discussion

Observations about the manual and the choices made

- **Figures fig1_1 and fig1_3 are not in the PDF.** They belong to the unit notes. The plots here are measured analogues: Figure 1 is the timing diagram (fig1_3 style)

and Figure 2 draws the three cost functions (fig1_1 style) from one response-time series.

- **Deadline reference differs between projects.** Project 1.1 defines the absolute deadline as actual release + D , so response time is measured from the actual release. Project 1.3 only says "a fixed assigned deadline". The code uses $D = T$ measured from the ideal release, so a late release eats into the slack. If the deadline were measured from the actual release, a stall before the release would never cause a miss at all.
- **Completion jitter needs a definition.** The manual says "analogously". The code takes the offset completion - ideal release, subtracts its mean, and reports the standard deviation and maximum absolute value of the result. Release jitter is actual release - ideal release, exactly as stated.
- **The hard utility is -100 after a miss.** The manual only says "large negative"; the value is arbitrary and is plotted on a symmetric-log axis so the 1 and 0 levels stay readable.
- **Results are non-deterministic.** Everything runs on a general-purpose OS inside a sandbox with uncontrolled background activity, in Python, with no timing guarantees. The numbers in this report come from one run and will differ on your machine and on a rerun. The structure of the results (tail larger than mean, jitter above zero, loaded worse than unloaded) is what the lab is about. Most sweeps use one run per setting, so some curves are not smooth.
- **The background thread competes through the GIL.** CPython lets only one thread run Python code at a time and switches every 5 ms by default, so the interference measured here is GIL plus OS scheduling, which is coarser than a real RTOS or ISR (see Section 4.5).

Manual's expected observations versus what was measured

Expected observation (Project 1.1)	Measured result
Tail clearly larger than the mean	Response time: mean 13.6 ms, 95th percentile 24.0 ms, maximum 49.3 ms

Real-Time Systems — Unit 1 Mini Project Report

Expected observation (Project 1.1)	Measured result
Release jitter small but non-zero when unloaded	Project 1.3 baseline release jitter std 0.075 ms; Project 1.1 unloaded std 0.853 ms, inflated by one job that overran its period
Miss fraction rises sharply once D drops below about the 95th percentile	On the measured R values: 5.0 % misses with D at the 95th percentile, 35.0 % with D at the mean (Figure 4)
Hard, firm and soft utilities differ only in consequence	Same 3 misses give mean utility -1.52 (hard), 0.975 (firm), 0.989 (soft) (Figure 2)

Files delivered

File	Purpose
rts_code/rt_common.py	Shared helpers: calibrated busy work, drift-free periodic loop, background-load thread, jitter and deadline analysis, hard/firm/soft utilities
rts_code/mp1_1_task_timing.py	Project 1.1, steps 1-9 and Lab Tasks 1-4
rts_code/mp1_2_wcet_tool.py	Project 1.2 measurement harness, statistics, margin, three targets, histograms
rts_code/mp1_3_jitter_analyzer.py	Project 1.3 baseline loop, three disturbances, sweeps, results table
rts_code/figures/*.png, results_*.json/csv	All sample-output figures used below and the raw result tables

Run each script with `python <file>.py` from inside `rts_code` (about 1 to 2 minutes each). Only NumPy and Matplotlib are needed. Run them one at a time: timing experiments disturb each other if run in parallel.

2. Mini Project 1.1: Real-Time Task Timing Behavior

2.1 Method

- **Job function (step 1).** `job(x)` burns a calibrated amount of CPU time: $8 + 8x$ ms on the common path for x in $[0, 1]$, plus $220(x - 0.9)$ ms (up to 22 ms) on a rarely taken long path when $x > 0.9$. Run time therefore depends on the input, as in Section 1.7. Calibration on this machine: about 14,900 loop iterations per millisecond (`busy_ms` in `rt_common.py`).
- **Isolated measurement (step 2).** 300 calls each, every call timed with `time.perf_counter()` immediately before and after. Two input sets: ordinary (x in $[0, 0.85]$, long path never taken) and adversarial ($x = 0.999$, longest path).
- **Periodic loop (steps 3-4).** $T = 40$ ms, $D = 0.8 T = 32$ ms, 120 jobs. `run_periodic` sleeps until the absolute instant $r_0 + k T$, recomputed every iteration, so errors cannot accumulate (no drift). Field inputs are uniform on $[0, 1]$, so the long path occurs in about 10 % of jobs.
- **Analysis (steps 5-6).** `analyze()` in `rt_common.py` computes release jitter, completion jitter, response time $R = \text{completion} - \text{actual release}$, and the hit flag $R \leq D$. Utilities: `hard = 1` or `-100`; `firm = 1` or `0`; `soft = 1` or $\exp(-(R - D)/\tau)$ with $\tau = 0.25 D$.

2.2 Step 2: execution time in isolation

300 calls	Mean (ms)	Std (ms)	Max (ms)	95th pct (ms)	Median (ms)
Ordinary inputs	12.90	4.27	29.79	22.03	11.97
Adversarial input ($x = 0.999$)	38.98	7.21	82.99	58.20	36.31

The ordinary maximum of 29.8 ms is labelled a measured, not proven, WCET estimate. The code path of the ordinary inputs cannot exceed about 14.8 ms ($8 + 8 \times 0.85$), so the measured maximum of 29.8 ms is about twice the real path cost: it is dominated by OS noise, which is exactly why a measured maximum is not a bound. A static analysis would derive the path bound from the code and a hardware model and would not depend on whether a test happened to be preempted. The adversarial input exposes 83.0 ms, 2.79 times the ordinary maximum, and its median (36.3 ms) matches the code's expected longest path of about 37.8 ms.

Real-Time Systems — Unit 1 Mini Project Report

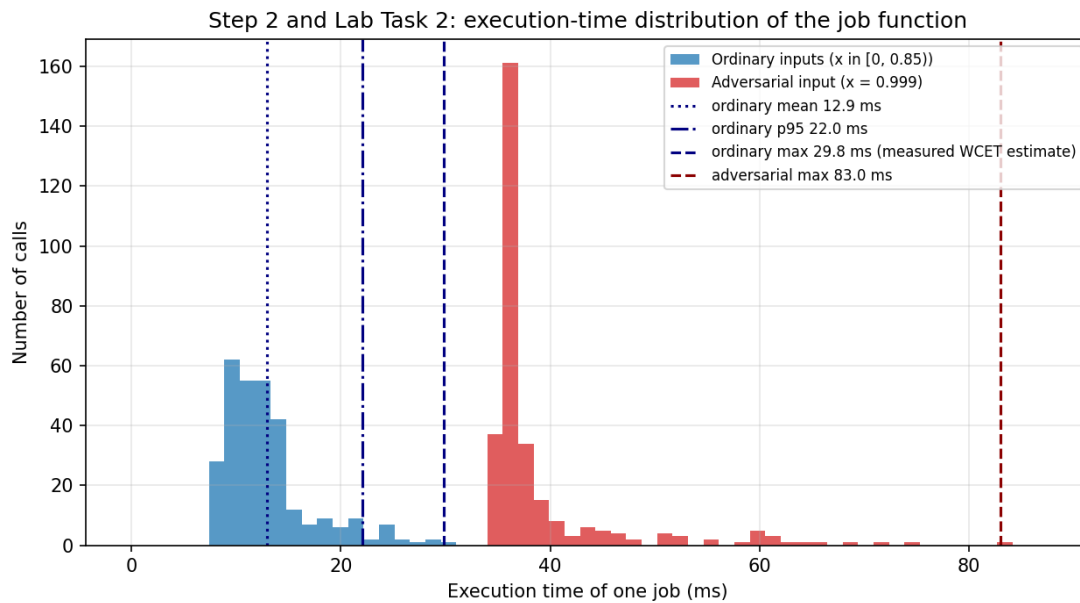


Figure 1. Execution-time distribution of the job function for ordinary and adversarial inputs

2.3 Steps 3-9: periodic run

Quantity	Value	Relative to $T = 40$ ms / $D = 32$ ms
Mean execution time (isolated)	12.9 ms	32.3 % of T , 40.3 % of D
Response time: mean / 95th pct / max	13.6 / 24.0 / 49.3 ms	max is 1.23 T and 1.54 D
Release jitter: std / max .	0.853 / 9.51 ms	2.13 % of T (std)
Completion jitter: std / max .	5.931 / 35.58 ms	14.8 % of T (std)
Deadline hits	117 of 120 (97.5 %)	3 misses
Mean utility: hard / firm / soft	-1.52 / 0.975 / 0.989	same three late jobs, different consequence

Real-Time Systems — Unit 1 Mini Project Report

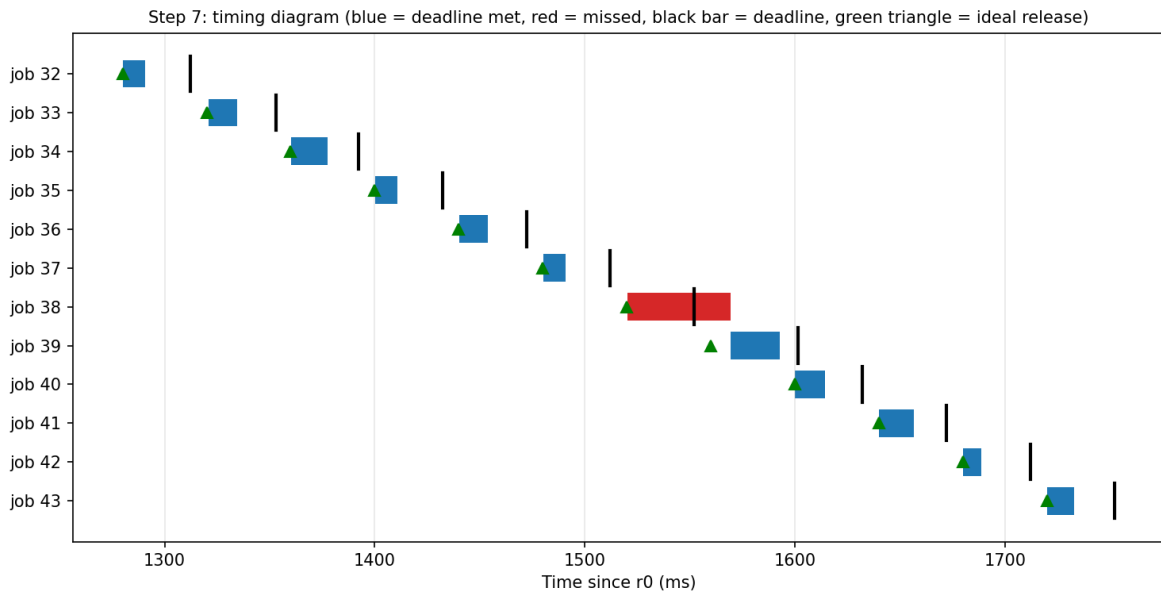


Figure 2. Step 7: timing diagram of 12 consecutive jobs around the slowest one (blue = met, red = missed, black bar = deadline, green triangle = ideal release)

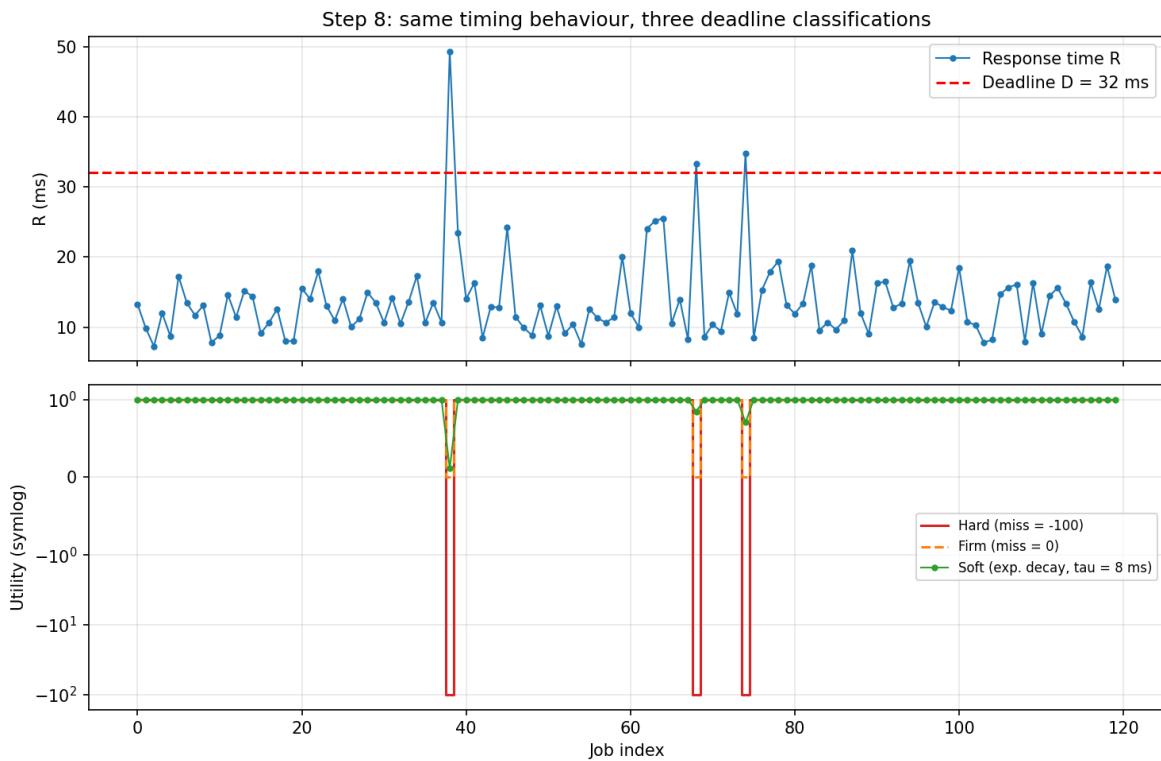


Figure 3. Step 8: response time against the deadline (top) and the three utility sequences from the same data (bottom)

Real-Time Systems — Unit 1 Mini Project Report

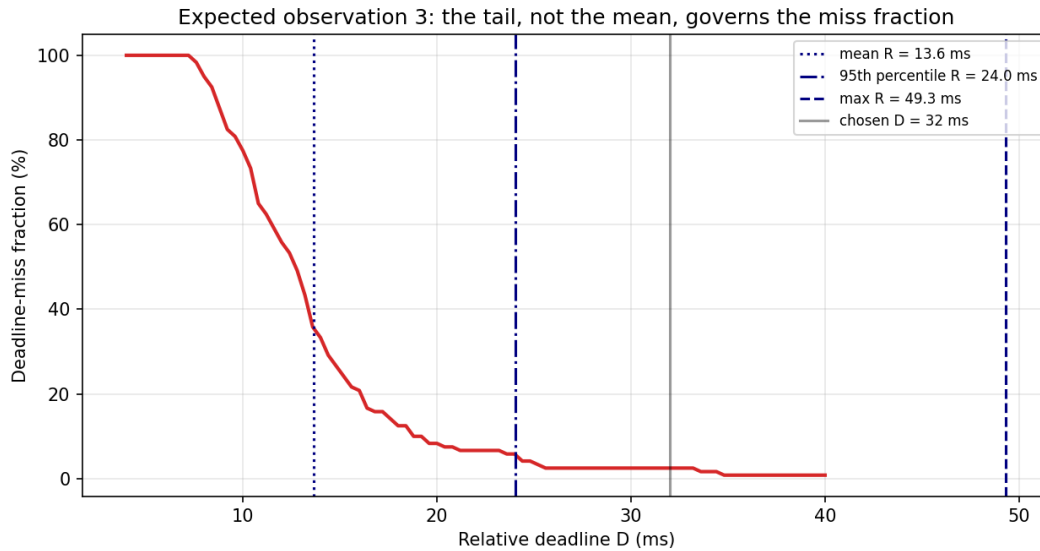


Figure 4. Miss fraction as the deadline is shrunk, evaluated on the measured response times

Reading Figure 4: with D at the mean response time, 35.0 % of jobs would miss; with D at the 95th percentile, 5.0 %. Below the 95th percentile the miss fraction climbs steeply, so the tail, not the mean, governs schedulability.

2.4 Lab tasks

Lab Task 1: background interference thread

	Release jitter std	Release max	Mean R	Max R	Hit fraction
Unloaded	0.853 ms	9.51 ms	13.6 ms	49.3 ms	97.5 %
With busy thread	10.396 ms	70.44 ms	25.5 ms	64.7 ms	80.0 %

A continuously busy thread raised the release jitter standard deviation by about 12 times (0.85 to 10.4 ms), roughly doubled the mean response time and dropped the hit fraction from 97.5 % to 80.0 %. The job and the thread share the interpreter, so while the job runs it gets only about half the CPU, and each time the loop wakes it may wait for the other thread's time slice. Longer response times then push later releases back, which feeds the jitter.

Real-Time Systems — Unit 1 Mini Project Report

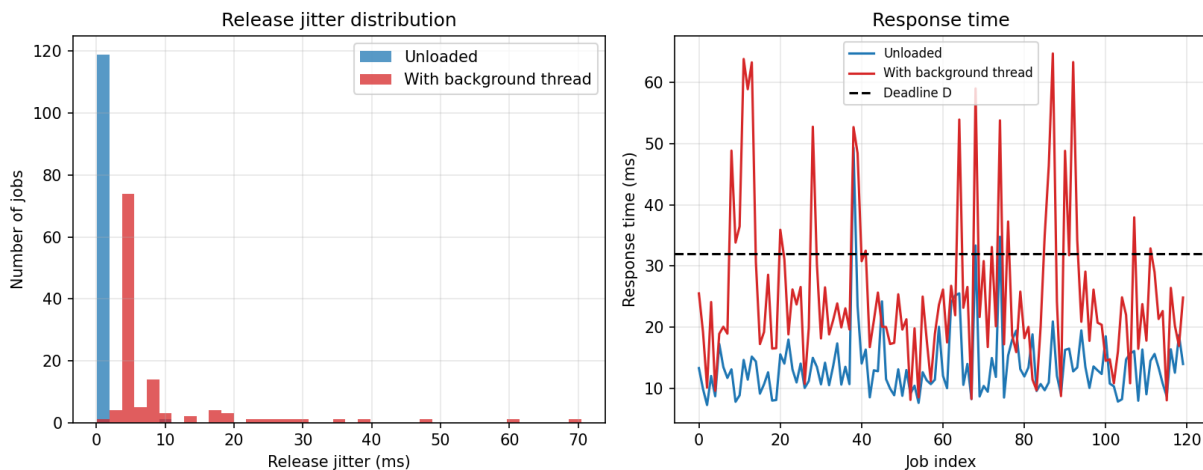


Figure 5. Lab Task 1: release-jitter histogram and response times, unloaded versus with the background thread

Lab Task 2: adversarial versus ordinary maximum

The adversarial maximum (83.0 ms) is 2.79 times the ordinary maximum (29.8 ms), and the medians differ by 3.0 times (36.3 versus 12.0 ms), which separates the real cost of the long path from noise in the maxima. Neither number substitutes for a sound static WCET bound (Section 1.7). The ordinary campaign never exercised the long path at all and its maximum was noise. The adversarial input only works because the author knows where the long path is; for real code that knowledge is exactly what is hard to guarantee. Both are single runs on one machine in one hardware state (cache, branch predictor, interference), and neither can prove that nothing slower exists. A static analyzer examines every feasible path and models the hardware, so its result is safe by construction, at the price of pessimism.

Lab Task 3: soft-deadline decay rate

Decay constant	Mean soft utility, unloaded run	Utility at $R = 1.5 D$
fast ($\tau = 0.05 D$)	0.980	0.000
default ($\tau = 0.25 D$)	0.989	0.135
slow ($\tau = 1.0 D$)	0.995	0.607

With only three late jobs, all of them barely late, the mean utility moves little between the three decay rates, so the decay curve only matters when lateness is large. The curves themselves differ greatly: a job 50 % late keeps almost no value

Real-Time Systems — Unit 1 Mini Project Report

under the fast decay and over 60 % under the slow one. The decay rate should therefore come from the application, not be chosen for convenience: for example, how late a video frame or telemetry sample can arrive before the user or the consumer stops benefiting, or the point at which the value has halved. A rate that is too slow hides real degradation; one that is too fast turns a soft task into a firm one.

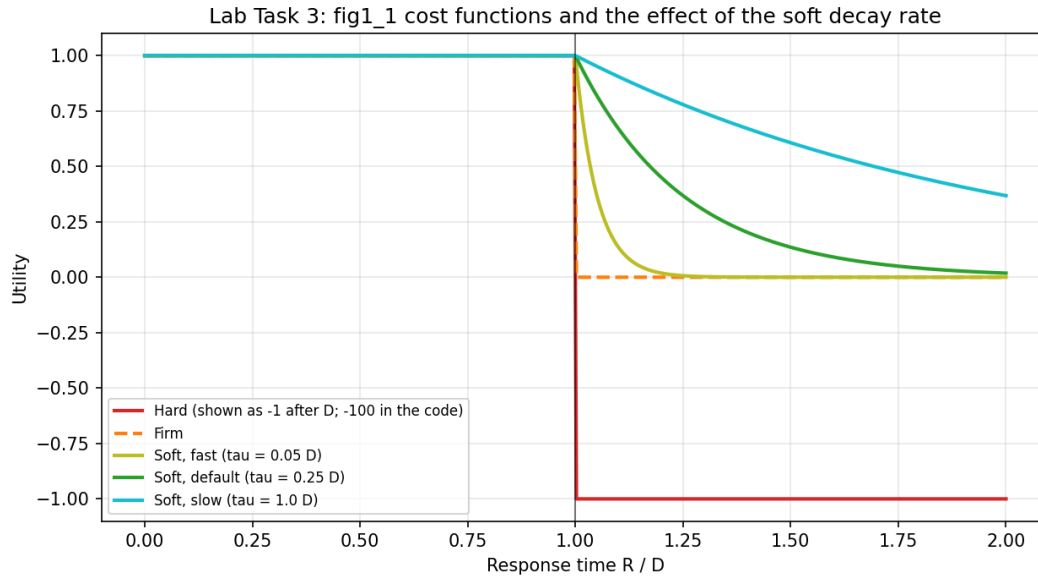


Figure 6. Lab Task 3: hard, firm and soft cost functions, with three decay constants for the soft case

Lab Task 4: vary T with $D = 0.8 T$

T (ms)	Release jitter std (ms)	% of T	Release max (ms)	Hit fraction	Jobs
20	17.355	86.78 %	75.85	64.0 %	100
40	0.223	0.56 %	1.93	96.0 %	100
80	0.051	0.06 %	0.38	99.0 %	100
160	1.802	1.13 %	18.31	100.0 %	100

The result has two regimes. At $T = 20$ ms the job's execution time (mean about 13 ms, up to about 38 ms on the long path) often exceeds the period, so jobs queue behind one another, releases slip, and jitter is 87 % of T with only 64 % of deadlines met; this is overload, not noise. At $T = 40$ ms and 80 ms the system is feasible and jitter is a fraction of a percent of T . At $T = 160$ ms a few isolated OS-noise spikes inflate the jitter (about 1 % of T), so jitter did not shrink steadily with T ; each setting is only 100

Real-Time Systems — Unit 1 Mini Project Report

jobs on a machine with uncontrolled background activity. The practical point is that the absolute jitter set by the platform does not shrink when the period shrinks. As T gets short, the same few hundred microseconds become a larger fraction of the period, and jitter matters even when every deadline is met: a sampled-data consumer that assumes uniform spacing sees irregular intervals, and output jitter makes downstream timing less predictable.

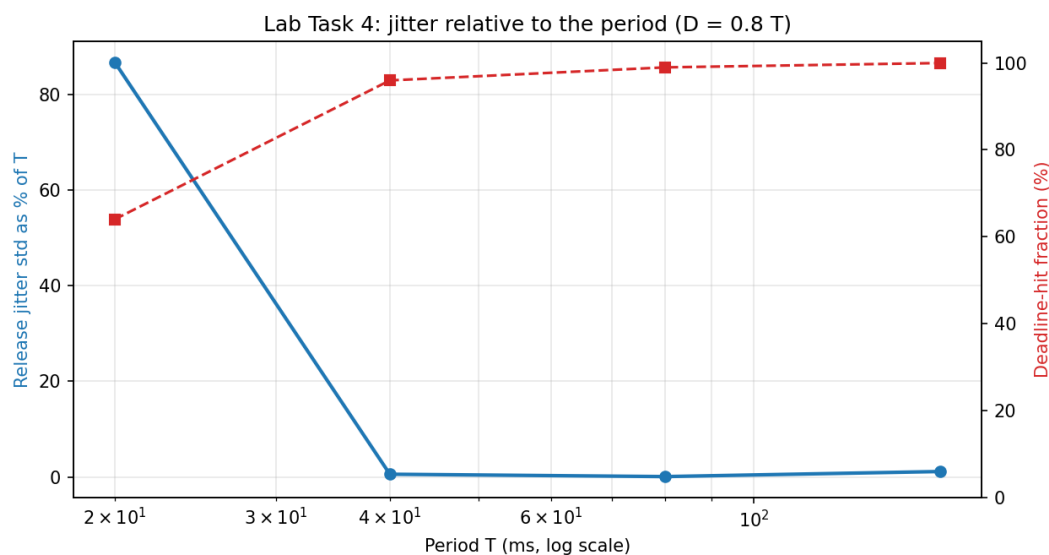


Figure 7. Lab Task 4: release jitter as a percentage of the period, and hit fraction, versus T

2.5 Step 9 summary

Execution time is 12.9 ms on average (32.3 % of T and 40.3 % of D) but its tail reaches about 29.8 ms in isolation and 49.3 ms as a response time, above both D and T . Release jitter had a standard deviation of 0.85 ms (maximum 9.5 ms, caused by one job that overran its period) and completion jitter 5.93 ms (maximum 35.6 ms), much larger because completion inherits the execution-time variation. 117 of 120 jobs (97.5 %) met the deadline.

Realistic cost function: this is a 25 Hz periodic job with variable processing time, like a video-frame or sensor-fusion update. A result that arrives after its deadline is of no use, but a late frame costs quality rather than safety, so a firm deadline is the most realistic classification, with soft behaviour acceptable if a slightly late result still has partial value. It would be a hard deadline only if the same job drove an actuator or safety function. (The manual's Section 1.1 application domains are not in the PDF, so this choice is argued from the task's behaviour; check it against your notes.)

3. Mini Project 1.2: Measurement-Based WCET Estimation Tool

3.1 Tool design

- **measure(func, inputs):** assumes nothing about the function. It runs 200 warm-up calls, disables garbage collection during timing, times each call with `time.perf_counter()` and keeps every sample.
- **summarize(samples):** mean, standard deviation, minimum, maximum and 90th, 95th and 99th percentiles, plus an outlier flag raised when the maximum exceeds twice the 99th percentile.
- **Safety margin:** `margin_from_utilization(0.65)` gives +53.8 %, so the margined WCET-for-scheduling is the observed maximum divided by 0.65, using the 60 to 70 percent utilization-budget convention. The margin is configurable.
- **Campaign:** runs 2000 random calls and 500 adversarial calls per target and reports both, plus the margined recommendation from the random campaign alone and from all samples.
- **Targets of increasing structure:** `linear_search` (early-exit branch; adversarial input = key absent, full scan), `collatz_steps` (data-dependent loop count; adversarial input $n = 77031$ with 350 steps, found by exhaustive offline search below 100,000), `packet_handler` (nested conditionals in a loop; adversarial input = valid control packet of 255 bytes that all take the deepest branch).

3.2 Comparison table (microseconds)

Function	Random mean	Random max	Adv. median	Adv. max	Adv. max > random max	Adv. median / random max	Margined, random only	Margined, all samples
<code>linear_search</code>	31.8	127.4	59.0	292.0	yes	0.46	196.0	449.3
<code>collatz_steps</code>	8.2	1295.1	22.2	109.1	no	0.02	1992.5	1992.5
<code>packet_handler</code>	4.0	63.8	135.0	502.4	yes	2.11	98.2	773.0

Margin is +53.8 % (65 % utilization budget). Outlier flag (maximum above twice the 99th percentile): `linear_search` random ok (1.5x), adversarial FLAGGED (2.8x); `collatz_steps` random FLAGGED (41.6x), adversarial ok (1.8x); `packet_handler` random ok (2.0x), adversarial FLAGGED (2.1x).

3.3 Histograms

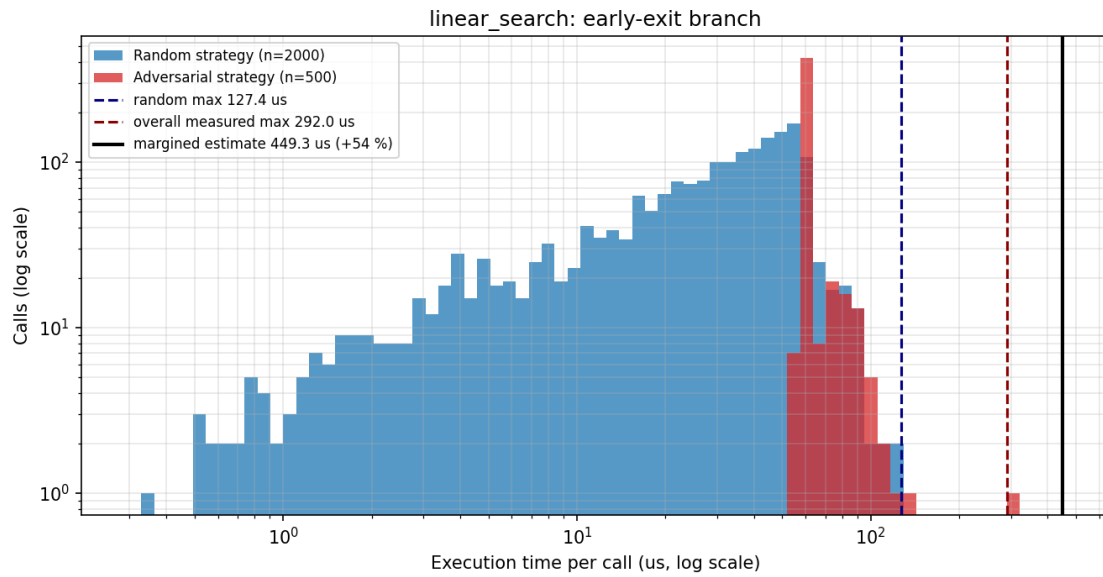


Figure 8. linear_search: random versus adversarial execution times, measured maximum and margined estimate (log axes)

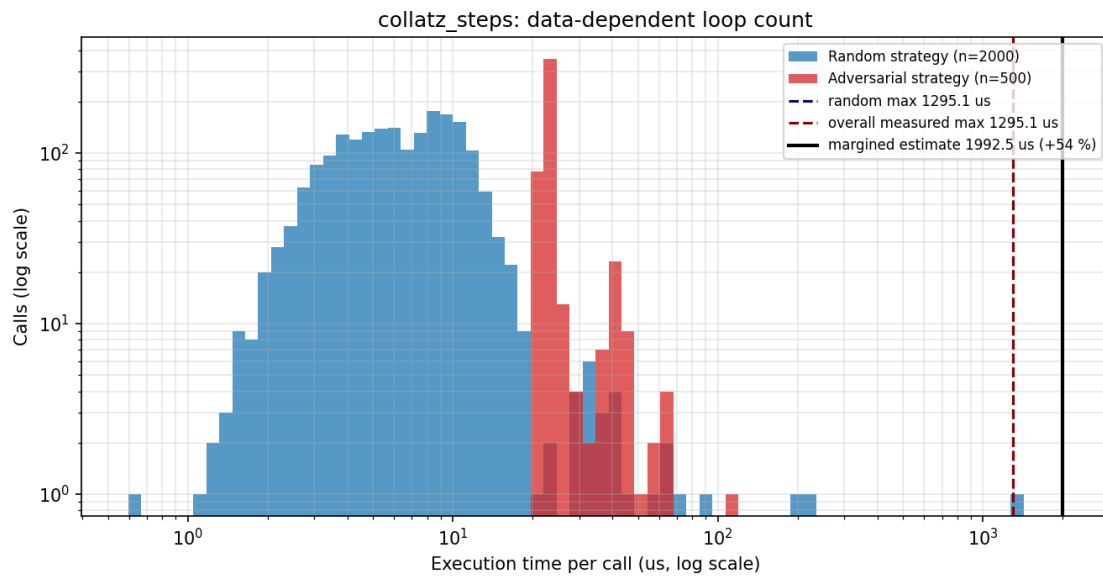


Figure 9. collatz_steps: random versus adversarial execution times, measured maximum and margined estimate (log axes)

Real-Time Systems — Unit 1 Mini Project Report

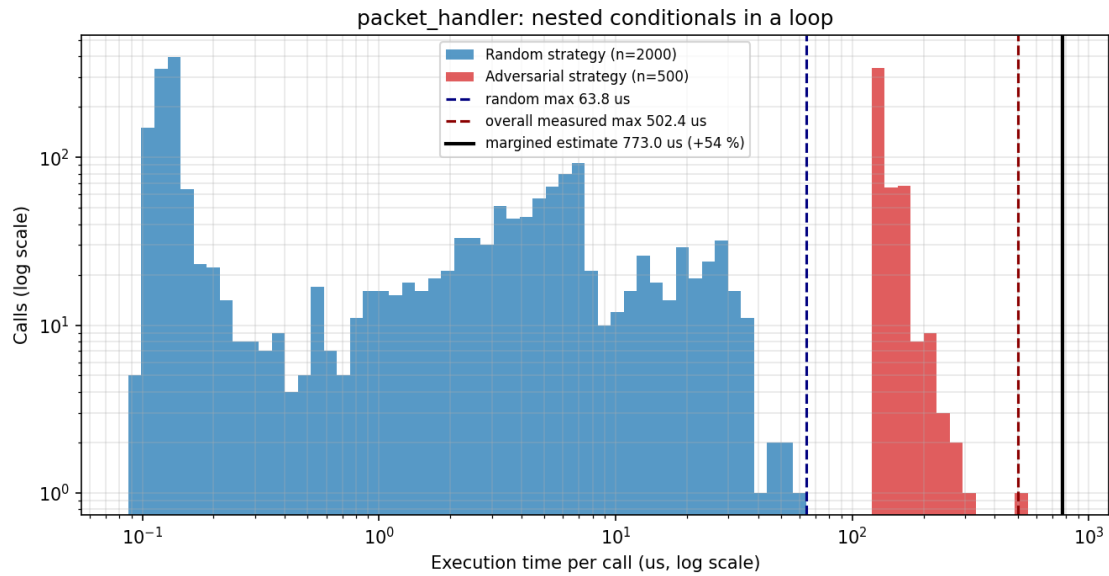


Figure 10. `packet_handler`: random versus adversarial execution times, measured maximum and margined estimate (log axes)

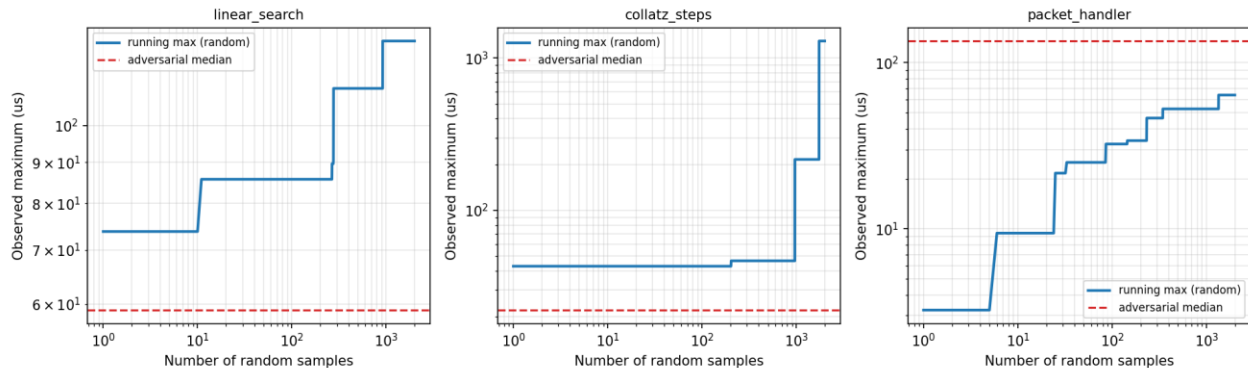


Figure 11. Running maximum of the random campaign against the adversarial median: more random samples do not reliably approach the worst path

3.4 Discussion

Soundness versus pessimism. Measurement is cheap and needs no model of the processor, but it is unsafe: the estimate is only the largest value that the campaign happened to see. A static analyzer is safe, because it bounds every feasible path on a model of the hardware, but it can be pessimistic and needs a good hardware model. The three targets show what this means in practice.

- **`packet_handler` was clearly underestimated by the random strategy.** The random campaign's maximum was 63.8 us and even with the +54 % margin the recommendation is only 98.2 us. The adversarial packet has a median of 135.0 us, 2.11 times the random maximum, and its margined estimate is 773.0 us. A random packet would need 255 bytes that all take the deepest branch, which

essentially never happens, so the random campaign cannot reach the worst path.

- **linear_search was estimated with reasonable confidence in path terms.**

Random keys that sit near the end of the array already cost about as much as a full scan, so random and adversarial distributions overlap (Figure 8) and the adversarial maximum (292.0 us) exceeds the random maximum (127.4 us). The remaining uncertainty is noise: the adversarial maximum is flagged as an outlier at 2.8 times its 99th percentile.

- **collatz_steps shows that a maximum can mislead in both directions.** The adversarial input (n = 77031) really is the worst path: its mean is about three times the random mean. But the random campaign's maximum of 1295.1 us is a single 41.6-times outlier above its 99th percentile, almost certainly OS preemption, not code. Because the margined estimate is built on the maximum, it becomes 1992.5 us, about 90 times the adversarial median. The tool flags this, and the flag is the point: an unexplained outlier needs investigation before anyone uses the number. Figure 11 shows the running maximum jumping in rare steps, so another thousand samples could move the estimate again.

Why a certification process would not accept this output alone (Section 1.8). The tool measures on one machine, one operating system and one hardware state, with no control over cache contents, branch-predictor history, interrupts or other tasks. It offers no proof that the campaign covered the worst path and no argument for why the true worst case lies below the margined value. The +54 % margin is a convention, not a statistical or analytic guarantee. The adversarial inputs were hand-derived by someone who knew the code, which does not scale to real software. Certification normally demands a safe bound from static analysis or from an argued, evidence-backed combination of methods; the tool's output is useful supporting evidence and a way to sanity-check an analyzer's bound, but it is not a certified WCET.

Limits of this experiment: timings of a few to a few hundred microseconds sit close to timer and OS noise, which is why several maxima are flagged; the sandbox's background noise was not controlled; and each campaign is a single run.

4. Mini Project 1.3: Real-Time Jitter Analyzer

4.1 Setup

Periodic task with $T = 10$ ms, a nominal job body of 2 ms of CPU, 250 jobs per experiment, drift-free release loop, and a fixed deadline $D = T = 10$ ms measured from the ideal release. Three disturbances can be switched on independently (experiment() in mp1_3_jitter_analyzer.py):

- **Background load:** BackgroundLoad thread burns CPU for a duty-cycle fraction of every 5 ms slice (duty 0.25, 0.5, 0.75, 1.0).
- **Random stall:** a Bernoulli trial before each job; on success the loop sleeps before the release is recorded. Swept over probability (0.02 to 0.4 at 4 ms) and magnitude (1 to 12 ms at probability 0.10).
- **Execution-time variability:** per-job workload drawn from a truncated normal (sigma 0.1 to 1.0 times C) or a heavy-tailed lognormal (sigma 0.25 to 1.5).

With every disturbance off, release jitter stays small and stable (std 0.075 ms, hit fraction 100 %), which is the control condition.

4.2 Results table

Configuration	Mechanism	Release std (ms)	Release max (ms)	Completion std (ms)	Completion max (ms)	Hit %
baseline	baseline	0.075	0.93	1.145	4.64	100.0
load duty=0.25	1 load	0.439	2.82	1.241	4.87	100.0
load duty=0.50	1 load	1.536	17.60	2.071	17.33	98.4
load duty=0.75	1 load	1.214	3.94	1.458	4.30	100.0
load duty=1.00	1 load	0.869	9.26	1.223	8.72	96.4
stall p=0.02, 4 ms	2a stall prob.	0.627	4.51	1.444	7.51	99.6
stall p=0.05, 4 ms	2a stall prob.	1.051	4.47	1.418	5.23	100.0

Real-Time Systems — Unit 1 Mini Project Report

Configuration	Mechanism	Release std (ms)	Release max (ms)	Completion std (ms)	Completion max (ms)	Hit %
stall p=0.10, 4 ms	2b stall mag.	1.239	4.40	1.573	5.67	100.0
stall p=0.20, 4 ms	2a stall prob.	1.675	5.03	1.918	5.71	100.0
stall p=0.40, 4 ms	2a stall prob.	2.014	4.49	2.246	4.87	100.0
stall p=0.10, 1 ms	2b stall mag.	0.339	1.41	1.114	3.42	100.0
stall p=0.10, 2 ms	2b stall mag.	0.644	2.42	1.256	4.06	100.0
stall p=0.10, 8 ms	2b stall mag.	2.490	9.55	2.615	11.88	90.4
stall p=0.10, 12 ms	2b stall mag.	4.049	20.14	4.199	17.61	88.0
normal sigma=0.10C	3a normal	0.029	0.29	0.792	3.62	100.0
normal sigma=0.25C	3a normal	0.146	1.80	1.034	4.48	100.0
normal sigma=0.50C	3a normal	0.310	5.05	1.393	12.87	99.6
normal sigma=1.00C	3a normal	0.321	4.28	2.677	11.31	97.6
lognormal sigma=0.25	3b lognormal	0.145	1.93	1.208	4.99	100.0
lognormal sigma=0.50	3b lognormal	0.147	1.76	1.294	5.24	100.0

Real-Time Systems — Unit 1 Mini Project Report

Configuration	Mechanism	Release std (ms)	Release max (ms)	Completion std (ms)	Completion max (ms)	Hit %
lognormal sigma=1.00	3b lognormal	2.070	17.64	4.580	23.20	88.4
lognormal sigma=1.50	3b lognormal	9.534	53.83	12.158	55.19	74.4

4.3 Sweep plots

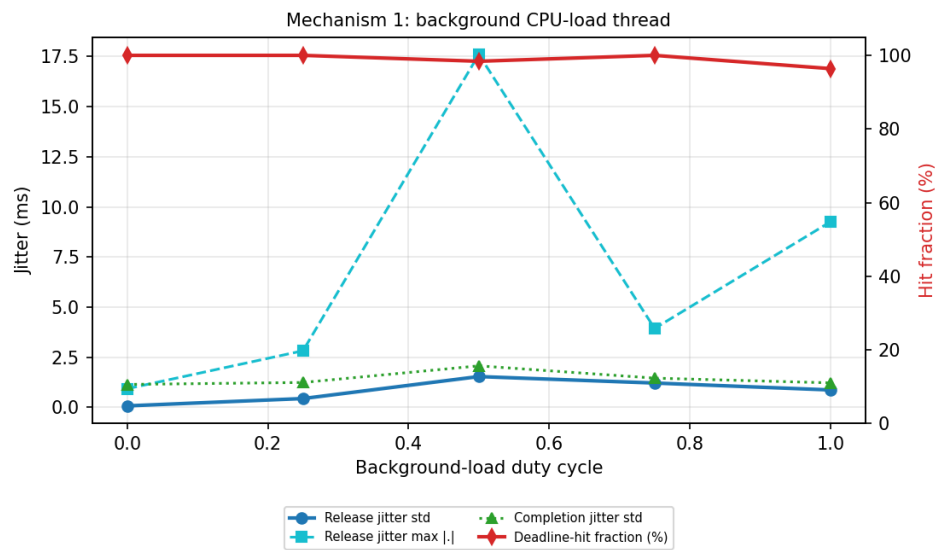


Figure 12. Mechanism 1: release jitter, completion jitter and hit fraction versus background-load duty cycle

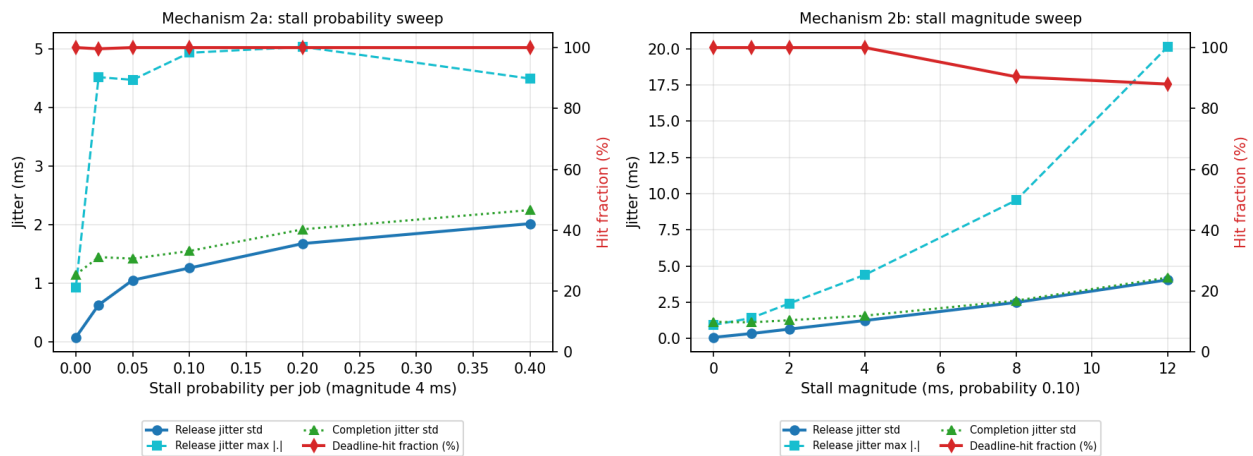


Figure 13. Mechanism 2: stall probability sweep (left) and stall magnitude sweep (right)

Real-Time Systems — Unit 1 Mini Project Report

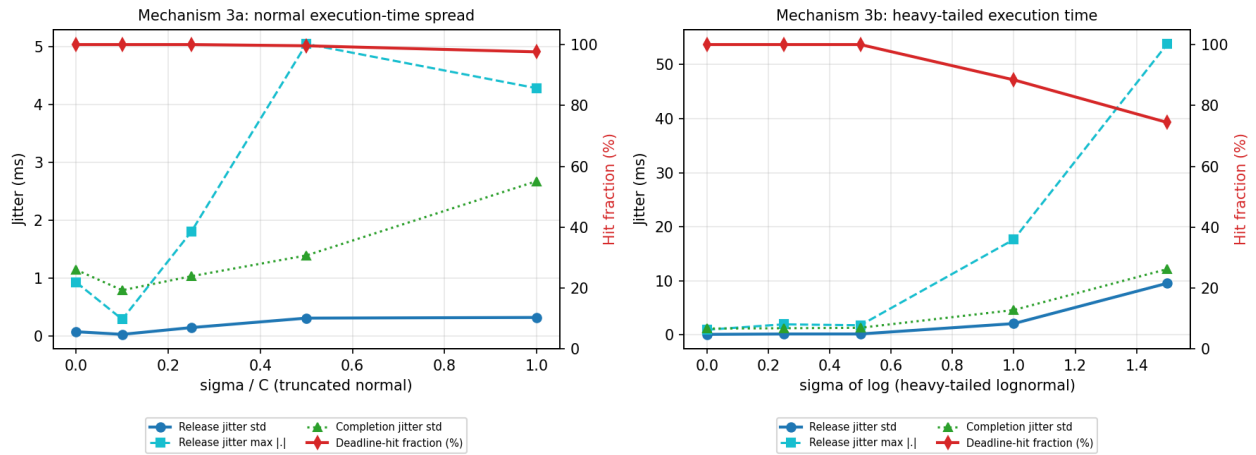


Figure 14. Mechanism 3: normal (left) and heavy-tailed lognormal (right) execution-time variability

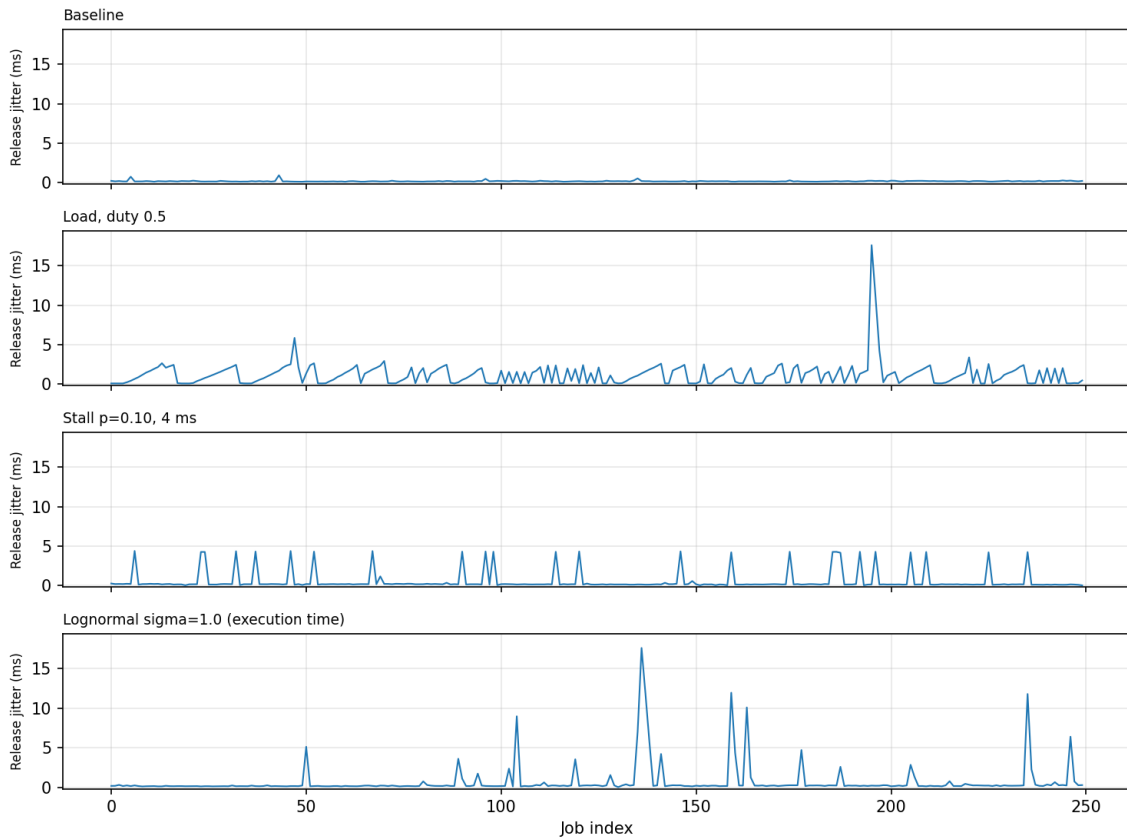


Figure 15. Release-jitter traces over 250 jobs for the baseline and one setting of each mechanism

4.4 Jitter growth without a drop in hit fraction

8 configurations kept a hit fraction of at least 99 % while their release-jitter standard deviation was at least 5 times the baseline (0.373 ms):

Real-Time Systems — Unit 1 Mini Project Report

Configuration	Release jitter std (ms)	Multiple of baseline	Hit fraction
load duty=0.25	0.439	6x	100.0 %
load duty=0.75	1.214	16x	100.0 %
stall p=0.02, 4 ms	0.627	8x	99.6 %
stall p=0.05, 4 ms	1.051	14x	100.0 %
stall p=0.10, 4 ms	1.239	17x	100.0 %
stall p=0.20, 4 ms	1.675	22x	100.0 %
stall p=0.40, 4 ms	2.014	27x	100.0 %
stall p=0.10, 2 ms	0.644	9x	100.0 %

The most extreme is a 4 ms stall with probability 0.4: release jitter is 27 times the baseline and every deadline is still met. Meeting every deadline and having low jitter are therefore distinct properties: the deadline $D = 10$ ms leaves 8 ms of slack beyond the 2 ms job, so the system absorbs the irregularity without a miss, but a consumer that expects releases every 10 ms sees them arrive up to several milliseconds early or late.

Real-Time Systems — Unit 1 Mini Project Report

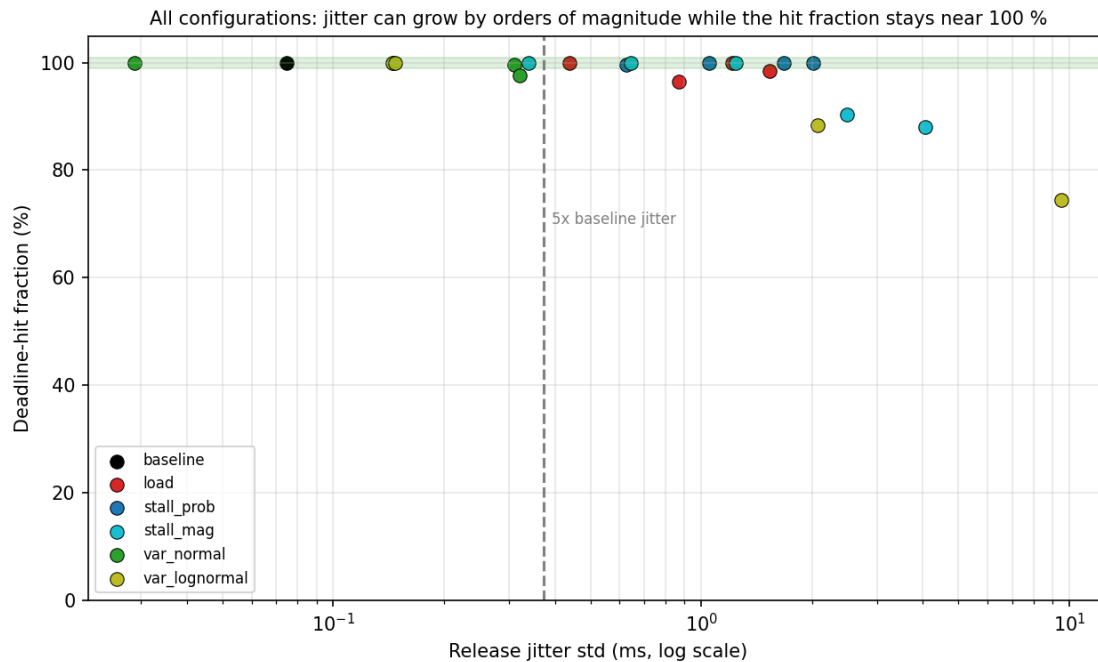


Figure 16. All configurations: release-jitter standard deviation versus hit fraction (log x-axis)

Further observations

- Execution-time variability acts on completion jitter far more than on release jitter. With a normal spread of 1.0 C, completion jitter std grows from 0.79 ms (sigma 0.1 C) to 2.68 ms while release jitter std stays at or below about 0.3 ms: the release loop is isolated from job length until jobs overrun the period.
- That isolation breaks when execution times become heavy-tailed. At lognormal sigma 1.5, long jobs overrun the 10 ms period, later releases queue behind them, and release jitter std jumps to 9.5 ms with the hit fraction falling to 74 %.
- Stall magnitude matters differently from stall probability. Raising the probability to 0.4 with 4 ms stalls raises jitter steadily and leaves the hit fraction at 100 %. Raising the magnitude to 8 and 12 ms makes the stall plus the 2 ms job approach or exceed D, so the hit fraction drops to 90 % and 88 %.
- The background-load curve is not monotonic (release std 1.54 ms at duty 0.5, 0.87 ms at duty 1.0). Each point is one run of 250 jobs; the 5 ms GIL switch interval is the same size as the 5 ms duty-cycle slice, so the interaction between the two is irregular. It should be repeated several times before reading a trend into it.

4.5 Discussion: mapping the disturbances to Section 1.6

Disturbance	Section 1.6 source approximated	Intended analogy	Where the analogy breaks down
Background CPU-load thread	General OS jitter and interference: scheduler preemption, other tasks competing for the core, shared-resource contention	Another activity takes CPU time at moments the task does not control, so the task wakes late and runs slower.	Python's GIL, with its 5 ms switch interval, dominates rather than an RTOS scheduler. There are no priorities or real-time scheduling policy, and the load is a regular duty cycle while real interference is bursty and state dependent.
Random pre-job stall (probability and magnitude)	Interrupts and DMA transfers that steal time or block resources at random moments	An occasional unpredictable event delays the job from becoming eligible, with a rate and a duration.	Real interrupts arrive at any instant, including in the middle of a job, nest, and are bounded by hardware priorities. Here the stall only happens at the release boundary and uses sleep, so the CPU stays free, whereas a DMA transfer stalls the memory bus and an ISR consumes CPU time.

Real-Time Systems — Unit 1 Mini Project Report

Disturbance	Section 1.6 source approximated	Intended analogy	Where the analogy breaks down
Per-job execution-time variability (normal or lognormal workload)	Cache effects and pipeline hazards that make the same code take different times depending on hardware state	Execution time varies from job to job around a nominal value, and a heavy tail models rare cache-miss-like long executions.	Real variation is state dependent and correlated: a cold cache after preemption, branch-predictor history, memory layout. Here each job is an independent random draw implemented by changing the amount of work, not by stalling the same instruction path.

Overall, the three mechanisms reproduce the qualitative effect of each Section 1.6 source (irregular release, irregular completion, occasional large delays) and make it controllable and repeatable, but they cannot reproduce the hardware mechanisms themselves. They support the qualitative lesson that non-determinism comes from several independent sources and that jitter and deadline hits must be measured separately, not a quantitative prediction for any real platform.

References

1. Real-Time Systems — Mini Projects, Unit 1 Lab Manual, prepared September 2026. Sections and procedures for Mini Projects 1.1, 1.2 and 1.3.
2. Python Software Foundation. Python Documentation — time module and `time.perf_counter()`.
3. Python Software Foundation. Python Documentation — threading module.
4. NumPy Developers. NumPy Documentation — arrays, random sampling and statistical functions.
5. Matplotlib Development Team. Matplotlib Documentation — plotting, histograms and timing diagrams.

GitHub Link

https://github.com/DerryCruise/Real_Time_System_MiniProject.git